Arm Robots with ROS: UR5e

M. J. Khojasteh

Fall 2026

1 Objective

This lab aims to explore the UR5e collaborative robotic arm, which is developed by Universal Robotics. This specific model is designed to be used for flexible industrial applications, research, and automation tasks. The UR5e features 6 degrees of freedom (DOF), allowing precise control over its movements.

Upon completion of this lab, students should be able to:

1. Configure and use `MoveIt` for motion planning and execution.
2. Create a DH Table and mathematically determine joint angles based on desired end-effector location
3. Directly control the robot trajectory by providing joint angles in simulation as well as on the real robot.

2 Pre-Lab

Please complete the pre-lab *Introduction to ROS2* before coming to lab.

3 Theory

3.1 Inverse Kinematics

Much of the theory behind this lab relies on the idea of inverse kinematics. Make sure you understand why and how we use inverse kinematics as many of the packages and frameworks that we will use in this lab boil down to this general idea. If you would like to check out a resource for this navigate to this website HERE

3.2 Denavit-Hartenberg (DH) Tables

3.2.1 Introduction

As we have discussed in class one of the easiest and most intuitive ways to calculate a frame transformation from our origin to our end effector is using the **DH-Table**. A general transformation between two frames can be represented with six parameters, consisting of the position and orientation of the frame with respect to one another. The DH-Framework allows us to use a reduced set of parameters that systematically transform the first frame into the other. This reduced set scales well for serial chains in manipulators that consist of many reference frames related by kinematic constraints (such as link lengths). The DH parameters are also a starting point on which the forward, inverse, or differential kinematics, as well as dynamics can be formulaically built. This framework is well accepted within the robotics community and a starting point for most explorations in this field.

Let's first consider two reference frames, the first we will call A, and the second we will call B. The transformation matrix between the two can be represented by ${}^{A}T_{B}$. As such we can represent our transformation frame as follows:

$$ {}^{A}T_{B} = \begin{bmatrix} {}^{A}R_{B} & {}^{A}\rho_{B} \\ 0 & 1 \end{bmatrix} \tag{1} $$

Where ${}^{A}R_{B} = R(\phi, \theta, \psi)$ is a 3x3 matrix that represents the rotation matrix from frame A to B and is a function of the roll (ϕ), pitch (θ), and yaw (ψ) angles. The 3x1 matrix ${}^{A}\rho_{B}$ is the matrix representation of the vector position of frame B with respect to A. This is where we move some distance in either x, y, or z from our frame in A to the new frame B.

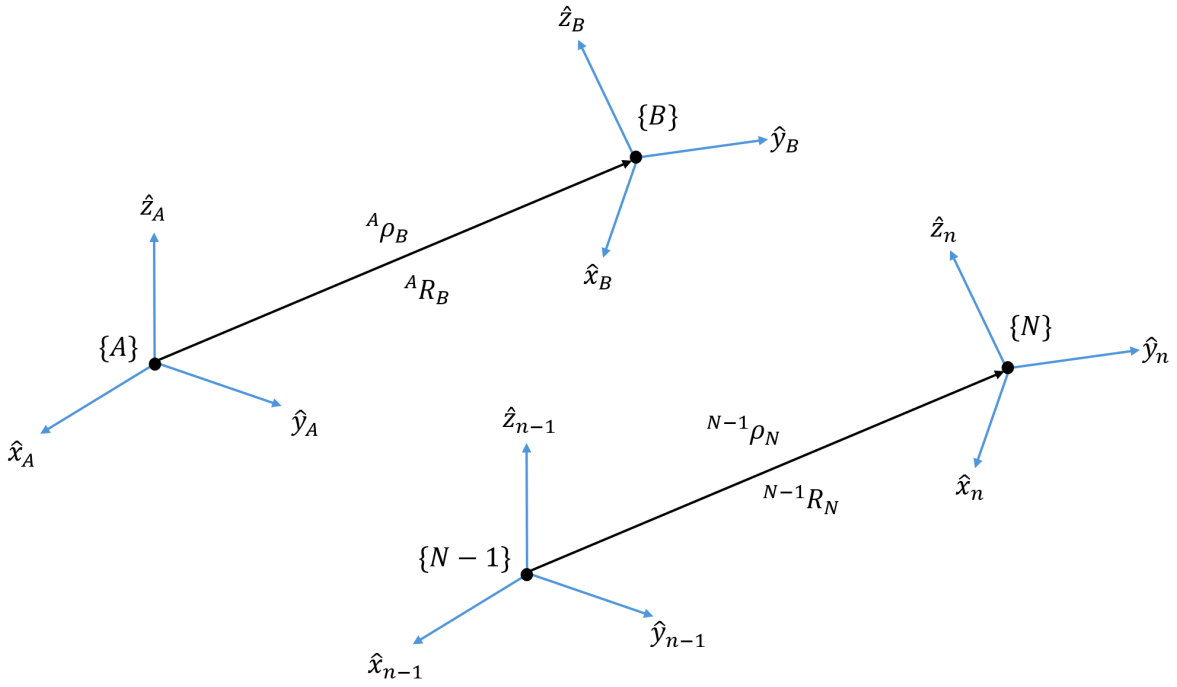


Figure 1: General Frame Transformation from Frame A to frame B

The DH framework provides a reduced set of 4 parameters to represent the position and orientation of a frame B with respect to frame A. For the rest of this theory segment we will focus on a particular frame of interest, frame N instead of frame B. Where it's position and orientation will be in regard to the previous frame $N-1$ instead of frame A, you can see this illustrated in Figure 1.

Keep in mind that the position of frames is not arbitrary with serial manipulators and is often the result of deliberate decision making. A clever choice of the frames of reference, as well as their relative position to each other contributes to the overall reduction in parameters.

3.2.2 How to Choose Frames

The reduction in parameters the the DH framework allows is made possible by making a few choices in regards to frame placement.

1. The joint axis at any given frame N is the z-axis of the frame N, that is $\hat{z}_n$. This means that as the link connecting frames $N-1$ and N moves with respect to frame $N-1$, it does so by either rotating about or translating along $\hat{z}_{n-1}$. Similarly, as the link connecting frames N and $N+1$ moves with respect to frame N it does so either rotating or translating along $\hat{z}_n$

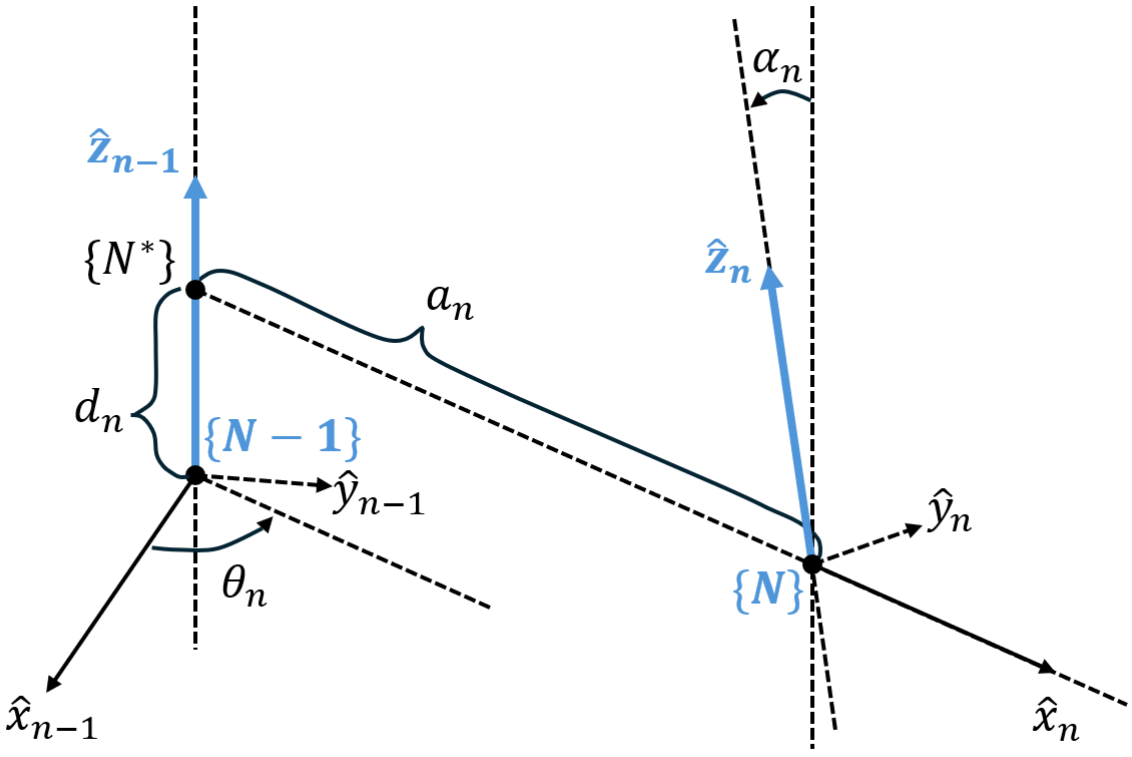


Figure 2: Frame Placement Choice #1

2. The position of frame N is chosen to lie at the intersection point of two axis. The first axis of these is the joint axis of frame N or $\hat{z}_n$. The second axis is along the common normal connecting the previous $N-1$ frame's joint axis ($\hat{Z}_{n-1}$) to the current N frame's joint axis ($\hat{z}_n$). For directionality, this second axis points along the common normal from the previous frame $N-1$ to the current frame N.

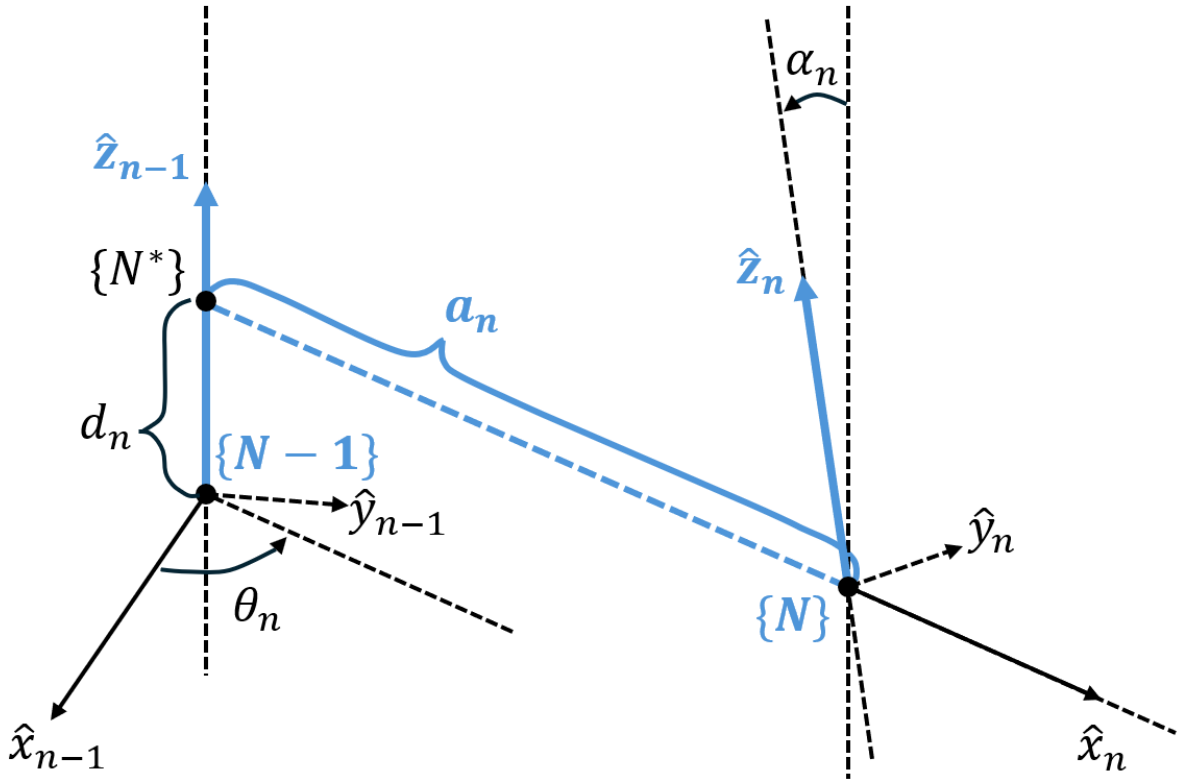


Figure 3: Frame Placement Choice #2

3. The $\hat{x}_n$ axis of the frame N is picked to point along the common normal from the previous frame $N-1$ to the current frame N. This is the same common normal picked in choice 2 above.

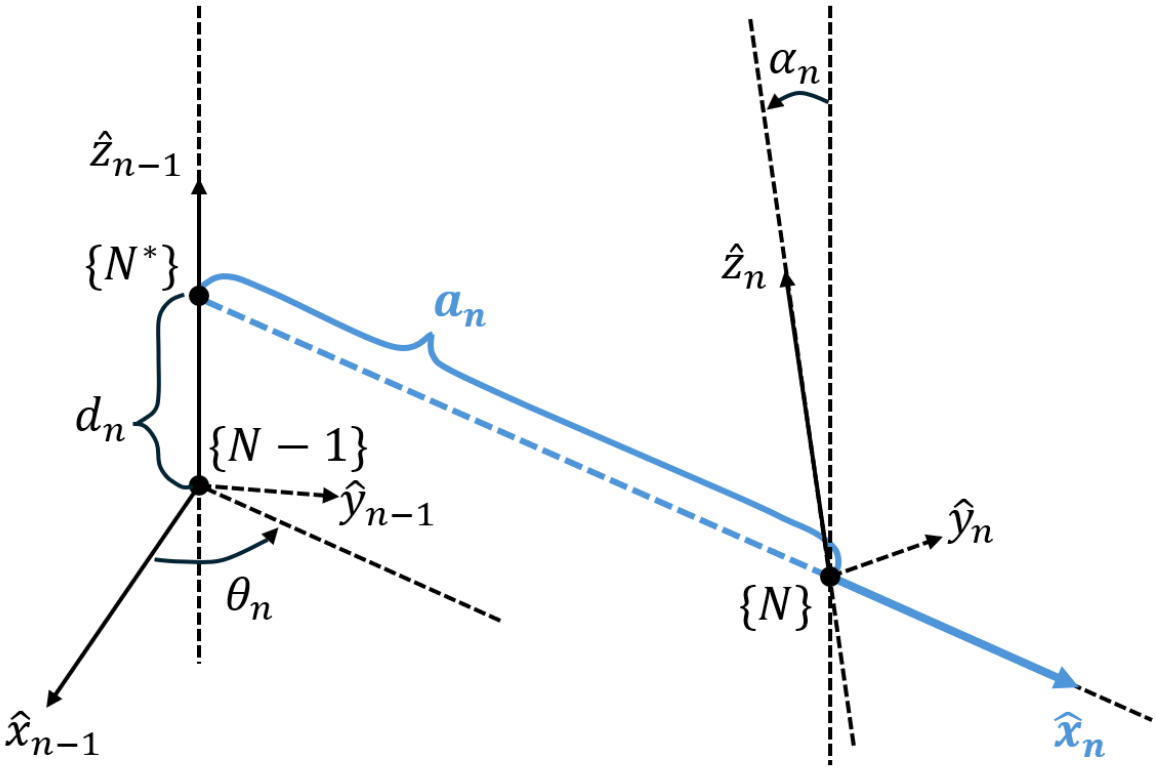


Figure 4: Frame Placement Choice #3

NOTE: There are a few variations of the DH notation arising from choices made in the two assumptions above. The version presented here is the standard DH notation.

3.2.3 DH Parameters

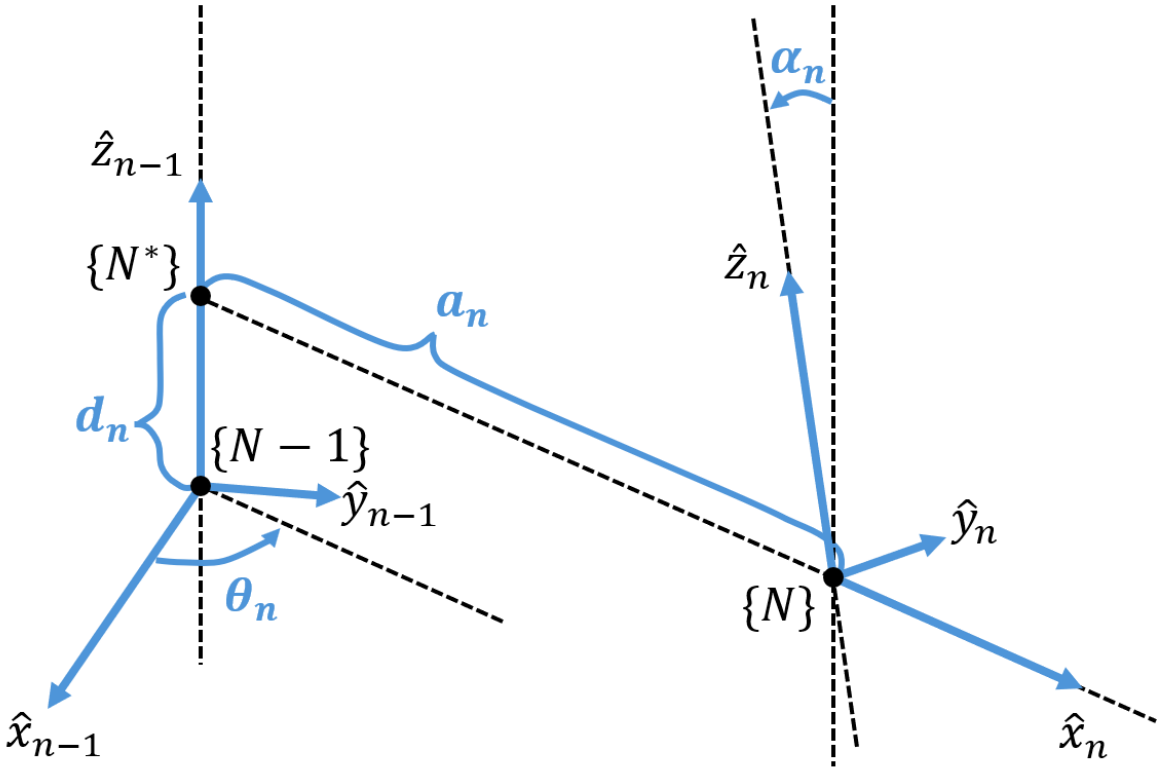


Figure 5: Frames of Interest Connected through 4 DH Parameters

From Figure 5 we can make a few observations, which are summarized below:

1. $\hat{z}_{n-1} \perp \hat{x}_{n-1}$ This is a natural consequence of any orthonormal frame
2. $\hat{z}_n \perp \hat{x}_n$ This is a natural consequence of any orthonormal frame
3. $\hat{z}_{n-1} \perp \hat{x}_n$ This is a consequence of picking $\hat{x}_n$ along the common normal

These observations lead to interesting conclusions:

1. A single rotation about $\hat{z}_{n-1}$ by an angle $\boldsymbol{\theta_n}$ gets us from $\hat{x}_{n-1}$ to $\hat{x}_n$
2. A single translation along $\hat{z}_{n-1}$ by a distance $\boldsymbol{d_n}$ gets us from the origin of $N-1$ to N^*
3. A single translation along $\hat{x}_n$ (now the same angle as $\hat{x}_{n-1}$) by a distance $\boldsymbol{a_n}$ gets us from the origin of N^* to N
4. A single rotation about $\hat{x}_n$ by an angle $\boldsymbol{\alpha_n}$ gets us from $\hat{z}_{n-1}$ to $\hat{z}_n$

Now we have a way to get from frame $N-1$ to frame N^* with two operations, shown above as step 1 and 2. Each involve the $\hat{z}_{n-1}$ axis. Then after reaching the $N*$ frame we can move to the N frame using the two operations shown in step 3 and 4, both involving $\hat{x}_n$. The four parameters, (θ_n, d_n, a_n, and α_n are collectively called the DH parameters and are

summarized in Table 1. The parameters a and α represent structural parameters of the linkage and assembly, whereas θ and d represent joint parameters for revolute and prismatic joints respectively.

Parameter	**Type**	**From**	**To**	**About / Along**	**Category**
θ_n	Rotation	$\hat{x}_{n-1}$	$\hat{x}_n$	$\hat{z}_{n-1}$	Joint
d_n	Translation	$\hat{x}_{n-1}$	$\hat{x}_n$	$\hat{z}_{n-1}$	Joint
a_n	Translation	$\hat{z}_{n-1}$	$\hat{z}_n$	$\hat{x}_n$	Structural
α_n	Rotation	$\hat{z}_{n-1}$	$\hat{z}_n$	$\hat{x}_n$	Structural

Table 1: DH Parameter Reference Table

The process of assigning DH frames and drawing a schematic is as follows:

1. Identify Joint axis J_1 through J_n for an N-joint manipulator. Assign axis $\hat{z}_{i-1}$ to joint axis J_i. This leads to standard and definite choices for $\hat{z}_0$ to $\hat{z}_{n-1}$. Pick $\hat{z}_n$ as required.

2. Find the common normal from $\hat{z}_{i-1}$ to $\hat{z}_i$ for each set of joints and mark intersection of this common normal and $\hat{z}_i$ as the origin of frame i. Pick the origin of frame 0 as required.

3. Assign $\hat{x}_i$ along the common normal identified above, at the origins picked above, in the direction from $\hat{z}_{i-1}$ towards $\hat{z}_i$. Pick $\hat{x}_0$ as required.

4. Complete the DH table of parameters.

Generally we find that the transformation between one frame to the next will take the following mathematical form:

$$
\begin{aligned}
{}^{i-1}T_i &= \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & 0 \\ \sin\theta_i & \cos\theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha_i & -\sin\alpha_i & 0 \\ 0 & \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned} \tag{2}
$$

Following from left to right we see the math follows the same steps we laid out previously. We align our x-axes, translate along the z-axis, translate along the common normal, and finally rotate about the x-axis to achieve the frame transformation.

3.3 UR5e Platform

The UR5e is considered a "cobot" which is simply a collaborative robot that works in the same space as humans. Some of the specifications of the UR5e arm can be seen below.



Figure 6: UR5e Robot

1. Reach: 850 mm / 33.5 in
2. Payload: 5 kg / 11 lbs.
3. Footprint: Ø 151 mm / 5.94 in
4. Weight: 18.4 kg / 45.4 lbs.
5. Repeatability: ±0.03 mm
6. Control Interface: ROS 2, Python, C++, URscript

4 Introduction

4.1 Setup

The setup is already done on the lab machines.

1. Install ROS 2 Humble on Ubuntu 22.04
2. Install the driver using the below command:

Terminal

```
$ sudo apt-get install ros-${ROS_DISTRO}-ur
```

4.2 Using MoveIt

In this section, you will learn how to control the UR5e robot using MoveIt, a powerful motion planning framework. You will use the MoveIt configuration for the UR5e and visualize the robot's movements in RViz.

4.2.1 Launch and configure the MoveIt Plugin

Terminal

```
$ ros2 launch ur_moveit_config ur_moveit.launch.py \
ur_type:=ur5e launch_rviz:=true
```

This command will launch the UR5e MoveIt configuration and start RViz, where you can visualize the robot's motions.

`ur_type:=ur5e`	`launch_rviz:=true` y
Specifies the robot type (in this case, UR5e)	This argument tells MoveIt to start RViz for visualization.

Follow these steps to configure the MoveIt plugin in RViz:

1. In RViz, navigate to the "Displays" panel and select "MotionPlanning."
2. Ensure that the **Robot Description** field is set to `robot_description`.
3. Set the **Planning Scene Topic** to `/monitored_planning_scene`. Click on the topic name to select from the drop-down menu.
4. Set the **Trajectory Topic** under **Planned Path** to `/display_planned_path`.
5. In the **Planning Request** section, change the **Planning Group** to `manipulator`. You can also see this in the MotionPlanning panel in the bottom left.

4.3 Understanding Robot Visualization

MoveIt provides four overlapping visualizations of the UR5e:

1. The robot's configuration in the `/monitored_planning_scene` planning environment (active by default).
2. The planned path for the robot (active by default).
3. Green: The start state for motion planning (disabled by default).
4. Orange: The goal state for motion planning (active by default).

The display states for each of these visualizations can be toggled on and off using checkboxes:

1. The planning scene robot using the **Show Robot Visual** checkbox in the **Scene Robot** tree menu.
2. The planned path using the **Show Robot Visual** checkbox in the **Planned Path** tree menu.
3. The start state using the **Query Start State** checkbox in the **Planning Request** tree menu.
4. The goal state using the **Query Goal State** checkbox in the **Planning Request** tree menu.

4.4 Interacting with the UR5e

For the next steps we will want only the scene robot, start state and goal state:

1. Check the **Show Robot Visual** checkbox in the **Planned Path** tree menu
2. Un-check the **Show Robot Visual** checkbox in the **Scene Robot** tree menu
3. Check the **Query Goal State** checkbox in the **Planning Request** tree menu
4. Check the **Query Start State** checkbox in the **Planning Request** tree menu

There should now be two interactive markers:

1. The marker corresponding to the orange arm represents the "Goal State" for motion planning.
2. The marker corresponding to the green arm represents the "Start State" for motion planning.

If you don't see the interactive markers press **Interact** in the top menu of RViz.

You can now drag the arm around to change its orientation and set different positions.

4.4.1 Detecting Collisions

1. Hide the planned path and goal state by disabling **Show Robot Visual** and **Query Goal State**.
2. Move the arm into a configuration where two links are in collision. The colliding links will turn red.
3. Enable **Use Collision-Aware IK** in the Planning tab of the MotionPlanning plugin.
4. Try moving the arm again. The IK solver will now attempt to prevent collisions.

4.4.2 Exploring Workspace Limits

1. Move the end-effector beyond its reachable workspace.
2. Observe how the system prevents unreachable positions.

4.5 Motion Planning with UR5e

1. Re-enable **Show Robot Visual** and **Query Goal State** before proceeding in this section.
2. Set a Start State position.
3. Set a Goal State position.
4. Ensure the states are not in collision with the robot itself.
5. Un-check the **Show Trail** checkbox in the Planned Path tree menu.
6. In the MotionPlanning window, under the Planning tab, click **Plan**.
7. Enable **Show Trail** in the Planned Path tree menu to visualize the generated motion path.

4.6 Inspecting Trajectories

1. Open Panels → Add New Panel.
2. Select Trajectory - Trajectory Slider.
3. Set a goal pose and run Plan.
4. Use the slider to inspect trajectory waypoints step by step.

4.7 Cartesian Path Planning

If the **"Use Cartesian Path"** checkbox is activated, the robot will attempt to move the end effector linearly in cartesian space.

Check out the tutorial this lab followed HERE if you want more info and/or a visual aid on the above topics!

4.8 Identifying Joint Angles for Pick-and-Place trajectory

Provided below in Table 2 are the measurements and dynamic limitations of the UR5e robot, you will use these given measurements to develop a DH table of the UR5e robot. Additionally you will find an image of the UR5e robot in which each of it's frames have been defined in Figure 7. The axis color guide is as follows:

1. X-Axis → RED
2. Y-Axis → GREEN
3. Z-Axis → BLUE

It is important that you follow these frame transformations as that is how MoveIt is setup as well.

Link Length	***Length*** [m]
$d[0]$	0.1625
$a[1]$	0.425
$a[2]$	0.3922
$d[3]$	0.1333
$d[4]$	0.0997
$d[5]$	0.0996

Table 2: Link Lengths for DH Table

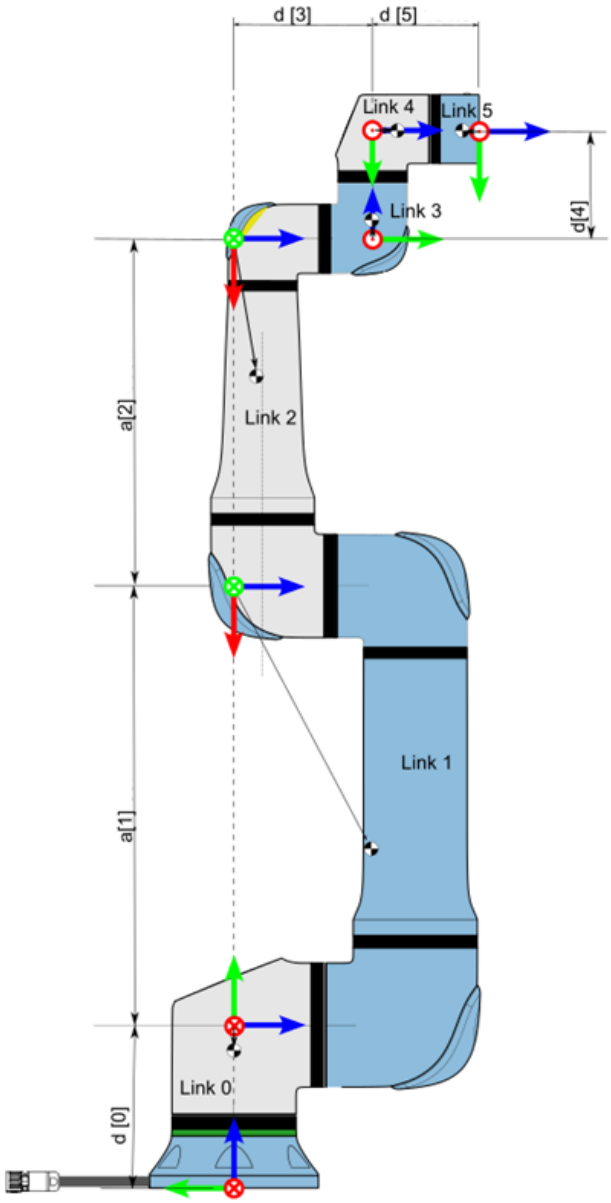


Figure 7: UR5e Robot with Defined Frames

Given the parameters as shown in Table 2 create a DH-table which includes the parameters a_i, d_i, α_i, and θ_i. Use this DH-table to define the frame transformation from frame 0 to 6. The matrix math should be "hand-derived" (i.e. using your DH table to formulate each frame transformation matrix and plug them into a calculator or software to solve for the total frame symbolic transformation matrix). You may also do this by hand however it will be time consuming. Make sure that your final answer is in terms of the θ of each joint.

Ideally from the forward kinematic frame transformation matrix we could derive the inverse kinematics, however this is a time consuming and non-trivial task. As such in the appendix you will find a document which goes through these calculations. Inverse kinematics allow you to find the joint angles necessary to reach a point in space, they are crucial for trajectory planning. Since the inverse kinematics are out of scope of this lab, we will skip this portion, but again refer to the appendix if you are interested.

5 Explore Pick and Place Setup - Simulation

The primary goal of this Lab is to control the robot's movements to perform a pick-and-place task, both in a simulation environment and on the actual robot. You will execute a sample movement script from the ur_robot_driver package and modify it to accommodate the specific joint angles you want to complete the pick-and-place task.

5.1 Launch the UR5e Simulator

For starting the driver there are two main launch files in the **ur_robot_driver** package.

```
ur_control.launch.py
```

- Starts ros2_control node including hardware interface, joint state broadcaster and a controller. This launch file also starts dashboard_client if real robot is used.

```
ur_dashboard_client.launch.py
```

- Starts the dashboard client for UR robots.

Also, there are predefined launch files for all supported types of UR robots.
The most relevant arguments given to the launch files are the following:

1. **ur_type** (mandatory) - a type of used UR robot (ur3, ur3e, ur5, ur5e, ur10, ur10e, or ur16e, ur20, ur30).
2. **robot_ip** (mandatory) - IP address by which the root can be reached. Use yyy.yyy.yyy.yyy for fake hardware.
3. **use_fake_hardware** (default: false) - use simple hardware emulator from ros2_control. Useful for testing launch files, descriptions, etc. See explanation below.
4. **initial_positions** (default: dictionary with all joint values set to 0) - Allows passing a dictionary to set the initial joint values for the fake hardware from ros2_control.
5. **fake_sensor_commands** (default: false) - enables setting sensor values for the hardware emulators. Useful for offline testing of controllers.
6. **robot_controller** (default: joint_trajectory_controller) - controller for robot joints to be started. Available controllers: joint_trajectory_controller, scaled_joint_trajectory_controller

To launch the simulator and visualize the robot in RViz:

Terminal

```
$ ros2 launch ur_robot_driver ur_control.launch.py \
ur_type:=ur5e robot_ip:=yyy.yyy.yyy.yyy use_fake_hardware:=true \
launch_rviz:=true
```

5.2 Sending commands to controllers

After launching the simulator, open another terminal to run controller commands.

Before running any commands, first check the controllers' state using:

Terminal

```
$ ros2 control list_controllers
```

Send some goal to the Joint Trajectory Controller by using a demo node from ros2_control_demos package by starting the following command in another terminal:

Terminal

```
$ ros2 launch ur_robot_driver \
test_scaled_joint_trajectory_controller.launch.py
```

After a few seconds the robot should move. Another way of sending commands to the control is through the usage of a python script:

Terminal

```
$ ros2 run ur_robot_driver example_move.py
```

You can find the github link to this script HERE

Create a new package and copy the `example_move.py` in it. Further, understand how the script is written, and modify it to meet the following criteria:

- A suitable Start State representing the initial position of the UR5e before picking an object. Make a note of the joint angles.
- A Goal State where the robot should place the object.
- Ensure that motion remains collision-free and within the robot workspace.
- Use Motion Planning to generate and visualize the trajectory.
- Execute the trajectory to complete the pick-and-place operation.

Demo this trajectory to the TA's for credit.

6 Explore Pick and Place Setup - Real Robot

To successfully control the UR5e robot using ROS 2, ensure the following network setup and execution steps are followed:

6.1 Network Configuration

The UR5e robot is connected to a router via Ethernet. The laptop running ROS 2 is connected wirelessly to the same router. This setup allows communication between the robot and the laptop over the network.

6.2 Verify Network Connectivity

Ensure both the robot and laptop are connected to the same network. To do this use the `ping` command to the robot's IP address and check if both the devices can communicate.

Terminal

```
$ ping <ip_address>
```

UR5e IP address: 192.168.131.143
Laptop IP address: 192.168.131.146

6.3 Start the UR5e ROS 2 Driver

On the UR5e laptop, run the following command to launch the UR5e drive:

Terminal

```
$ ros2 launch ur_robot_driver ur_control.launch.py \
ur_type:=ur5e robot_ip:=192.168.131.143 use_fake_hardware:=false \
launch_rviz:=true
```

- This starts the ros2_control node and connects to the UR5e robot.

6.4 Configure the Robot in External Control Mode

On the UR5e teach pendant:

- Create a new program
- Add an External Control node
- Make sure the IP address of the laptop is set properly in the External Control node settings
- Save and start the program
- Run the Example Script on the Laptop

In a new terminal on the laptop, execute the example motion script:

Terminal

```
$ ros2 run ur_robot_driver <your_script_name>.py
```

6.5 Observe Robot Execution

The UR5e should begin executing the predefined movements from the script. If the robot does not move, verify:

- The External Control program is running on the teach pendant
- The network connection is stable
- The correct IP addresses are used in both steps

Algorithm 0: Pseudocode for a Safe Pick and Place Routine (UR5e)

Initialize ROS2 node;
Declare parameters (controller, joints);
Create action client for trajectory control;
Create service client for MoveIt IK;
Define helper to compute IK for a target pose;
Convert orientation from Euler (r, p, y) to quaternion (x, y, z, w) before sending IK request;
Define helper to send a joint trajectory;
Define a safe "home" pose;

Function `safe_pick_and_place(`*pick_pose, place_pose*`):`

    Move to home;
    Move above pick → down to pick → grasp;
    Return home;
    Move above place → down to place → release;
    Return home;

In main: initialize node, define pick/place poses,
call `safe_pick_and_place()`, then shutdown node;

Working with the real platform

- **Check devices connected to the router:**

```
arp -a
```

- **Verify that the robot is reachable on the network:**

```
ping 192.168.131.143
```

- **Step 1:** Launch the Universal Robots control node.

```
ros2 launch ur_robot_driver ur_control.launch.py \
    ur_type:=ur5e \
    robot_ip:=<your_robot_ip> \
    use_fake_hardware:=true \
    launch_rviz:=false
```

- **Step 2:** Launch the MoveIt configuration for motion planning.

```
ros2 launch ur_moveit_config ur_moveit.launch.py \
    ur_type:=ur5e \
    launch_rviz:=true
```

- **Step 3:** Run your custom pick-and-place node.

```
ros2 run ur5e_pick_place <your_script>
```

7 Deliverables

1. Show basic movement and trajectory planning.
2. DH Table with resulting frame transformations.
3. Show the pick and place algorithm in simulation and on the real platform.

8 Questions

1. Describe the process of planning and executing trajectory using MoveIt. What challenges did you face, and how did you resolve them?
2. What could this lab improve on, where do you wish explanation was better?

9 Report

Make sure to submit a tech memo which follows the skeleton/guidelines that are located in MyCourses.

Your report should include the following:

1. DH-Table
2. Hand calculations
3. Relevant figures
4. Pre-Lab deliverables
5. Answers to all of the questions.

10 Appendix

As stated previously in the lab, inverse kinematics is a non-trivial task. Typically anything beyond 3-DOF becomes an exercise in linear algebra and analysis that is not in the scope of this lab. As such the document below is provided and contains the solutions to the forward and inverse kinematics of the UR5e robot (Page 13 and 23 respectively).

Kinematic Solutions to the UR5e Robot

Additionally it should be noted that the solution guide uses a different convention of the DH table than what is taught in class. In class the convention is to rotate around z, translate along z, translate along x, and then rotate about x. This puts your DH table in the order of θ, d, a, and α. The provided solution guide uses what is called "Craig's Convention" which differs in order of frame transformations. Using Craig's Convention you translate on x, rotate on x, translate on z, and finally rotate about z. This puts your DH table in the order of a, α, d, and θ. This allows us to generate the frames as shown below and in the solution guide.

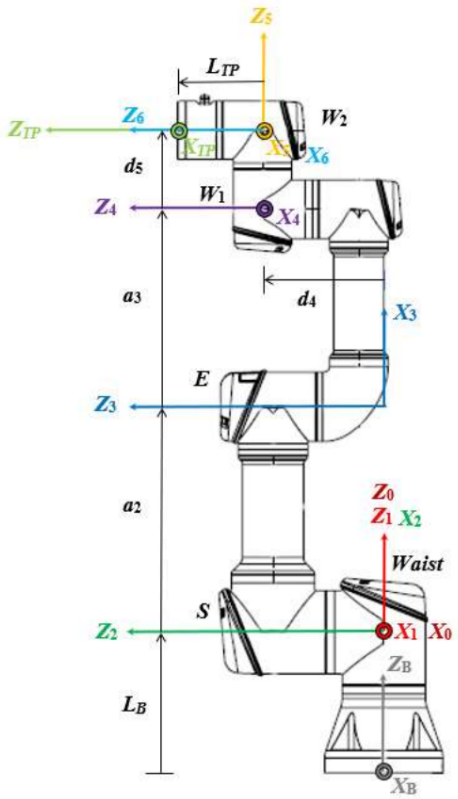


Figure 8: UR5e Robot with Defined Frames using Craig's Convention